Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»
(Университет ИТМО)

Факультет программной инженерии и компьютерной техники
Образовательная программа «Веб-технологии»

О Т Ч Ё Т

по дисциплине «Базы данных и знаний»

Тема: Проектирование и анализ схемы хранения композитных объектов, представляемых на основе pure operation-based CRDT

Обучающийся: Иванов Никита Денисович

Группа: P4110

Срок выполнения: апрель 2026

Санкт-Петербург
2026

Содержание

ВВЕДЕНИЕ **4**

1 ПОСТАНОВКА ЗАДАЧИ И МОДЕЛЬ ДАННЫХ **5**

1.1 Состав репозитория 5

1.2 Фактическая структура операции 5

2 РЕАЛИЗАЦИЯ ВАРИАНТА НА POSTGRESQL **7**

2.1 Назначение PostgreSQL-варианта 7

2.2 Логическая и физическая схема 7

2.3 Целостность и нормализация 8

2.4 Основные сценарии запросов 9

2.5 Фрагмент SQL-схемы 10

2.6 Преимущества и ограничения 11

3 РЕАЛИЗАЦИЯ ВАРИАНТА НА MONGODB **12**

3.1 Назначение MongoDB-варианта 12

3.2 Выбранная документная модель 12

3.3 Индексация и сценарии чтения 13

3.4 Преимущества и ограничения 14

3.5 Фрагмент инициализации MongoDB 14

3.6 Наблюдения по экспериментальным данным 16

4 ИЗМЕРЕНИЯ И РЕЗУЛЬТАТЫ **17**

4.1 Набор команд для анализа 17

4.2 Результаты PostgreSQL 17

4.3 Результаты MongoDB 18

5 ВАРИАНТ НА YTSAURUS **20**

6 СРАВНЕНИЕ ПОДХОДОВ **21**

ЗАКЛЮЧЕНИЕ **22**

ПРИЛОЖЕНИЕ А. СВОДНЫЕ РЕЗУЛЬТАТЫ POSTGRESQL **23**

ПРИЛОЖЕНИЕ Б. СВОДНЫЕ РЕЗУЛЬТАТЫ MONGODB **25**

ПРИЛОЖЕНИЕ В. СРАВНЕНИЕ ПОДХОДОВ ХРАНЕНИЯ **27**

ВВЕДЕНИЕ

В рамках индивидуального задания рассматривается задача проектирования хранилища для композитных объектов, состояние которых определяется журналом операций. Исходная предметная область связана с прототипом коллаборативного календарного приложения, поддерживающего оффлайн-режим, локальные изменения и последующую синхронизацию между репликами.

Актуальность темы обусловлена тем, что при построении систем на основе CRDT необходимо не только определить прикладную модель конфликтов и синхронизации, но и подобрать такую структуру хранения, которая будет одновременно удобной для записи операций, восстановления истории объекта и анализа производительности типовых запросов.

Целью работы является сравнение двух вариантов хранения одной и той же модели операций: в реляционной СУБД PostgreSQL и в документной СУБД MongoDB.

Для достижения поставленной цели решаются следующие задачи:

- определить логическую структуру данных, описывающую реплики, объекты и операции;
- спроектировать схему хранения в PostgreSQL;
- спроектировать схему хранения в MongoDB;
- проанализировать сильные и слабые стороны каждого подхода;
- подготовить основу для дальнейшего анализа индексов и планов выполнения запросов.

ГЛАВА 1. ПОСТАНОВКА ЗАДАЧИ И МОДЕЛЬ ДАННЫХ

Репозиторий выделен как отдельная инфраструктурная часть индивидуального задания и предназначен для исследования нескольких вариантов хранения одного и того же контракта операций CRDT. Практический интерес работы состоит не только в выборе СУБД, но и в сравнении разных способов организации одного и того же operation log.

1.1 Состав репозитория

В текущем состоянии репозиторий включает следующие основные части:

- каталог `postgresql` с `Docker Compose`, миграцией схемы и материалами по анализу запросов;
- каталог `mongodb` с `Docker Compose`, инициализацией коллекции и материалами по `executionStats`;
- каталог `report` с исходным текстом данного отчёта.

Таким образом, работа организована как воспроизводимое исследовательское окружение, в котором каждая СУБД поднимается изолированно и имеет собственные скрипты инициализации.

1.2 Фактическая структура операции

Согласно описанию в `README.md` и специализированных подкаталогах, центральной единицей прикладной модели является операция со следующими полями:

- `opId`;
- `txId`;

- `objectId`;
- `replicaId`;
- `clock`;
- `action`.

Поле `clock` хранит причинно-следственную информацию. Поле `action` содержит полезную нагрузку операции. В обоих вариантах используется одна и та же прикладная структура; различается только физическая организация данных.

ГЛАВА 2. РЕАЛИЗАЦИЯ ВАРИАНТА НА POSTGRESQL

В каталоге `postgresql` реализовано автономное окружение для реляционного моделирования журнала CRDT-операций. Запуск выполняется через `dockercompose-fpostgresql/compose.yamlup-d`, а физическая схема создаётся миграцией `postgresql/initdb/001_schema.sql`.

2.1 Назначение PostgreSQL-варианта

PostgreSQL используется как базовая реляционная СУБД для хранения append-only operation log. Такой вариант удобен для анализа, поскольку позволяет явно задать структуру данных, внешние ключи, ограничения целостности и индексную стратегию, а затем исследовать поведение запросов через `EXPLAIN ANALYZE`.

2.2 Логическая и физическая схема

Текущая реляционная схема включает три таблицы:

- `replicas` — справочник логических реплик;
- `objects` — справочник корневых CRDT-объектов;
- `operations` — неизменяемый журнал операций.

Связи между сущностями заданы явно: одна реплика может породить много операций, один объект может иметь много операций, каждая операция принадлежит ровно одной реплике и ровно одному объекту. При этом отдельная таблица зависимостей между операциями не вводится, поскольку причинность задаётся содержимым поля `clock`.

Физическая схема повторяет логическую модель почти напрямую. Идентификаторы хранятся как `TEXT`, что соответствует строковому формату

библиотеки. Поля `clock` и `action` сохраняются в `JSONB`, поскольку они имеют полуструктурированную природу и на текущем этапе исследования не декомпозируются в набор отдельных реляционных таблиц.

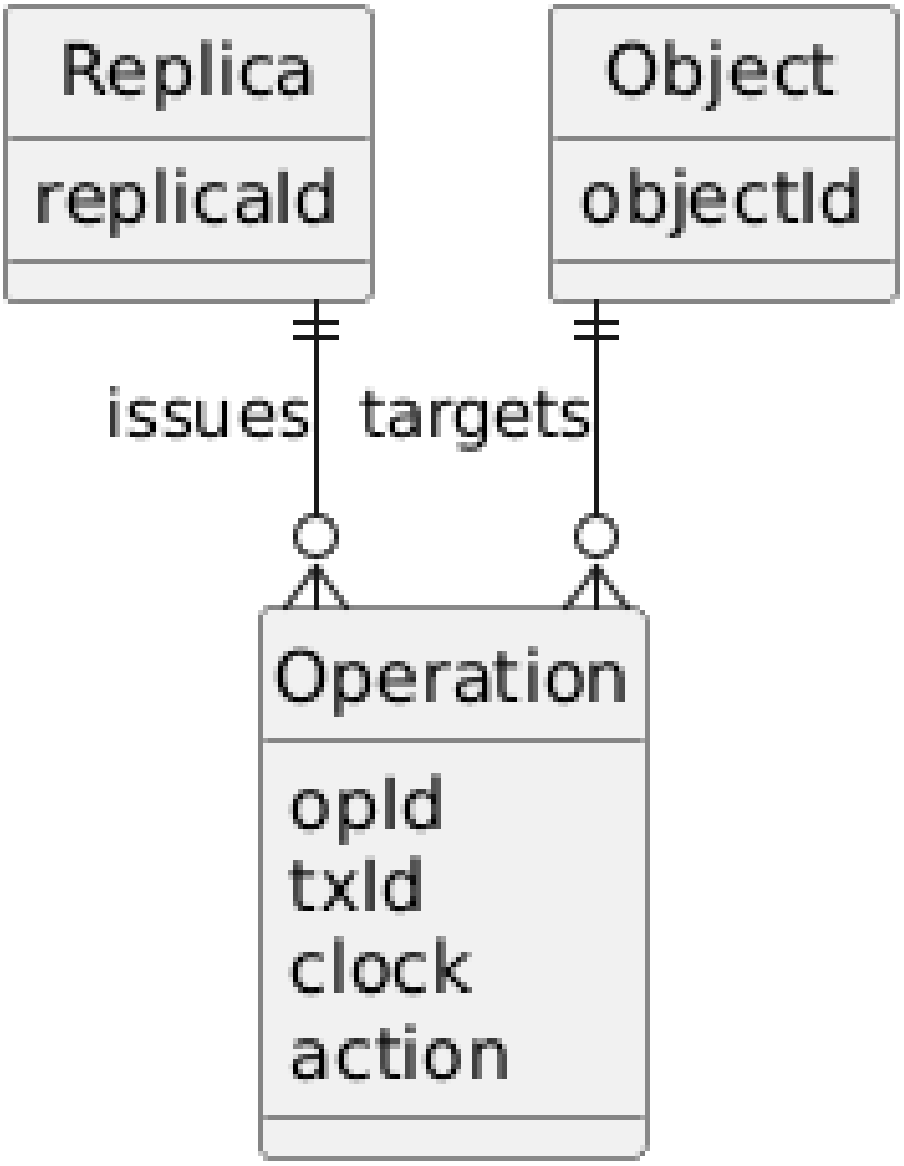

Рисунок 1 – Логическая ER-диаграмма варианта хранения в PostgreSQL

2.3 Целостность и нормализация

Схему PostgreSQL корректно считать частично нормализованной. Каркас из таблиц `replicas`, `objects` и `operations` соответствует обычной реляционной модели и обеспечивает:

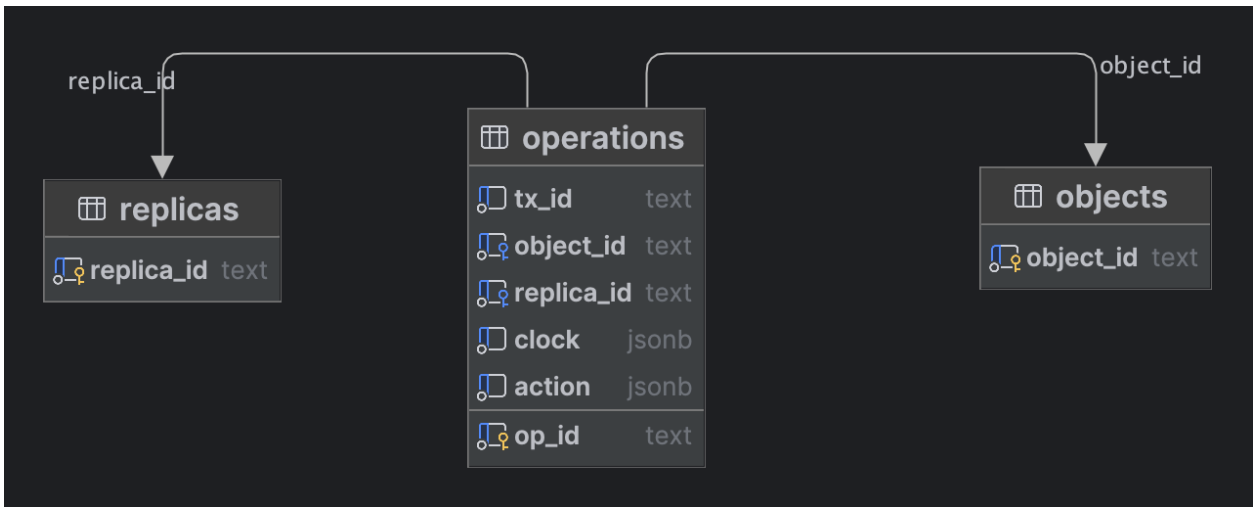

Рисунок 2 – Физическая схема таблиц PostgreSQL

- первичные ключи для всех основных сущностей;
- внешние ключи из `operations` в `replicas` и `objects`;
- `CHECK`-ограничения, контролирующие, что `clock` и `action` являются JSON-объектами;
- проверку наличия поля `type` внутри `action`.

При этом доменная корректность содержимого `clock` и `action` остаётся обязанностью прикладного уровня. Это означает, что PostgreSQL отвечает за структурную и ссылочную целостность, а библиотека — за корректную семантику CRDT-операций.

2.4 Основные сценарии запросов

Из `postgresql/README.md` видно, что в реляционном варианте в фокусе находятся выборки по операциям как самостоятельным записям. Базовые сценарии:

- получение полной истории операций для заданного `objectId`;
- получение операций конкретной транзакции по `txId`;
- получение операций, выпущенных конкретной репликой;

- чтение сохранённых векторных часов для конкретной операции;
- фильтрация по типу действия внутри `action`.

Этот набор запросов показывает, что PostgreSQL ориентирован не только на чтение истории одного объекта, но и на глобальный анализ журнала операций.

2.5 Фрагмент SQL-схемы

Ниже приведён фактический фрагмент миграции, создающей основные таблицы и индексы реляционного варианта:

```sql
CREATE TABLE replicas (
    replica_id TEXT PRIMARY KEY
);

CREATE TABLE objects (
    object_id TEXT PRIMARY KEY
);

CREATE TABLE operations (
    op_id TEXT PRIMARY KEY,
    tx_id TEXT NOT NULL,
    object_id TEXT NOT NULL REFERENCES objects(object_id) ON DELETE CASCADE,
    replica_id TEXT NOT NULL REFERENCES replicas(replica_id),
    clock JSONB NOT NULL,
    action JSONB NOT NULL,
    CONSTRAINT chk_clock_is_object CHECK (jsonb_typeof(clock) = 'object'),
    CONSTRAINT chk_action_is_object CHECK (jsonb_typeof(action) = 'object'),
    CONSTRAINT chk_action_has_type CHECK (action ? 'type')
);

CREATE INDEX idx_operations_object_id
    ON operations (object_id);

CREATE INDEX idx_operations_replica_id
    ON operations (replica_id);

```

```sql
CREATE INDEX idx_operations_tx_id
    ON operations (tx_id);

CREATE INDEX idx_operations_action_type
    ON operations ((action ->> 'type'));

CREATE INDEX idx_operations_clock_gin
    ON operations USING GIN (clock);

CREATE INDEX idx_operations_action_gin
    ON operations USING GIN (action);
```

Листинг 1 – Миграция схемы PostgreSQL

2.6 Преимущества и ограничения

К основным преимуществам PostgreSQL-варианта относятся строгая схема хранения, прозрачная ссылочная целостность, удобство аналитических выборок по `opId`, `txId` и `objectId`, а также естественная поддержка append-only журнала как множества независимых строк.

Ограничения реляционного варианта также явно видны. Во-первых, часть структуры операции остаётся внутри `JSONB`, поэтому схема является лишь частично нормализованной. Во-вторых, длинная история одного объекта требует сортировки и может становиться заметной по стоимости. В-третьих, низкоселективные широкие выборки могут приводить к менее эффективным планам, включая `Seq Scan`.

ГЛАВА 3. РЕАЛИЗАЦИЯ ВАРИАНТА НА MONGODB

В каталоге `mongodb` размещено автономное окружение документной СУБД MongoDB. Запуск выполняется через `dockercompose-fmongodb/compose.yamlup-d`, а структура коллекции и индексов задаётся скриптом `mongodb/initdb/001_init.js`.

3.1 Назначение MongoDB-варианта

MongoDB рассматривается как объектно-центричная альтернатива PostgreSQL. Если в реляционном варианте единицей хранения является отдельная операция, то здесь основной единицей становится документ объекта со встроенным массивом операций.

3.2 Выбранная документная модель

Для текущего этапа принята модель “один объект — один документ со встроенной историей операций”. В текущем варианте используется одна коллекция:

- `objects`.

Документ верхнего уровня содержит:

- `objectId`;
- массив `operations`.

Каждый элемент массива `operations` соответствует библиотечному типу `Operation` и содержит те же поля, что и в реляционном варианте. Таким образом, меняется не состав данных, а единица агрегирования на уровне хранилища.

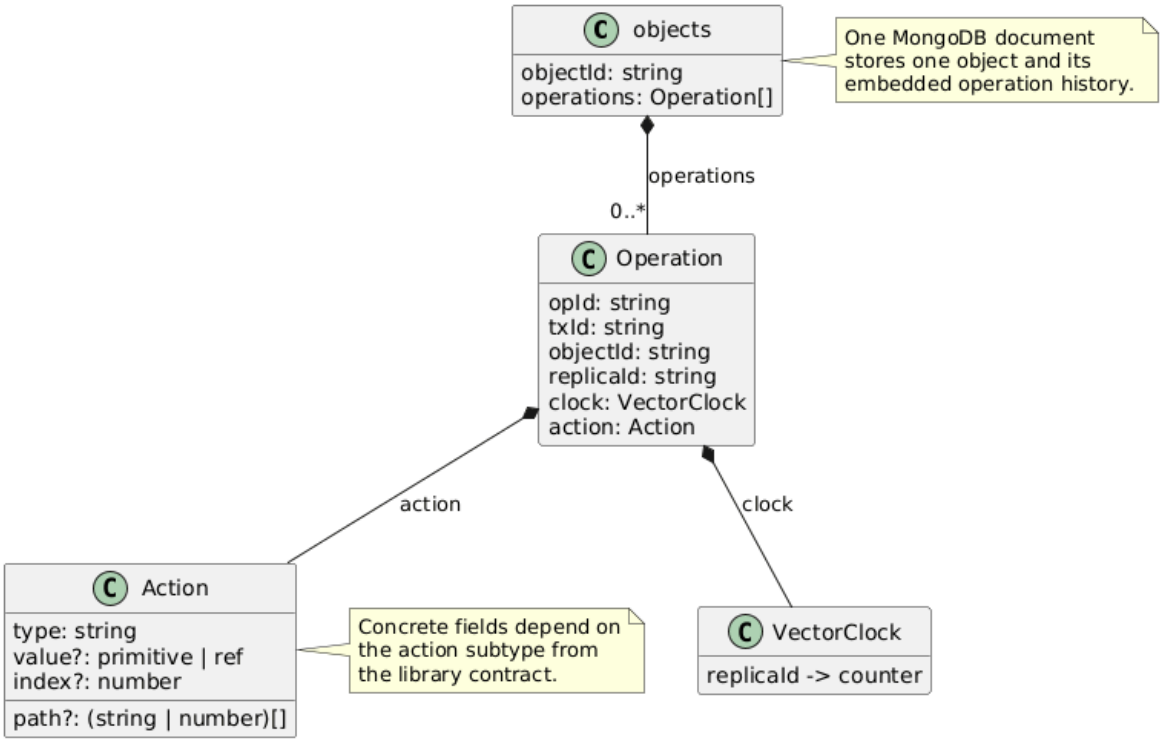

Рисунок 3 – Структура документа объекта в MongoDB

3.3 Индексация и сценарии чтения

В `mongodb/README.md` зафиксированы следующие направления индексации:

- уникальный индекс по `objectId`;
- индекс по `operations.opId`;
- индекс по `operations.replicaId`;
- индекс по `operations.action.type`.

Это хорошо отражает специфику документной модели: индексируются вложенные поля массива операций, а не top-level коллекция операций. Основные сценарии запросов при такой организации следующие:

- поиск документа объекта по `objectId`;
- чтение полной встроенной истории конкретного объекта;

- поиск объекта, содержащего операцию с заданным `opId`;
- поиск объектов, в истории которых есть операции конкретной реплики;
- поиск по значениям `operations.action.type`.

3.4 Преимущества и ограничения

Сильные стороны MongoDB-варианта заключаются в естественном хранении полуструктурированных полей `clock` и `action`, а также в том, что полная история конкретного объекта может быть прочитана из одного документа без отдельного журнала top-level операций.

Ограничения модели также существенны. Размер документа MongoDB ограничен `16 MB`, очень длинная история объекта ухудшает устойчивость embedded-модели, а глобальные выборки по операциям через весь набор документов менее естественны, чем в PostgreSQL. В `README` также отмечено, что при дальнейшем росте истории может потребоваться сегментация документа.

3.5 Фрагмент инициализации MongoDB

Ниже приведён фактический фрагмент скрипта, создающего коллекцию `objects` и основные индексы документного варианта:

```
db = db.getSiblingDB("crdt_lab");

db.createCollection("objects", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["objectId", "operations"],
      properties: {
        objectId: { bsonType: "string" },
        operations: {
          bsonType: "array",
          items: {
```

```
13            bsonType: "object",
14            required: [
15              "opId",
16              "txId",
17              "objectId",
18              "replicaId",
19              "clock",
20              "action"
21            ],
22            properties: {
23              opId: { bsonType: "string" },
24              txId: { bsonType: "string" },
25              objectId: { bsonType: "string" },
26              replicaId: { bsonType: "string" },
27              clock: { bsonType: "object" },
28              action: {
29                bsonType: "object",
30                required: ["type"],
31                properties: {
32                  type: { bsonType: "string" }
33                }
34              }
35            }
36          }
37        }
38      }
39    }
40  }
41 });
42
43 db.objects.createIndex({ objectId: 1 }, { unique: true, name: "uq_object_id"
    });
44 db.objects.createIndex({ "operations.opId": 1 }, { name: "idx_operations_op_id
    " });
45 db.objects.createIndex({ "operations.replicaId": 1 }, { name: "
    idx_operations_replica_id" });
46 db.objects.createIndex({ "operations.action.type": 1 }, { name: "
    idx_operations_action_type" });
```

Листинг 2 – Скрипт инициализации MongoDB

3.6 Наблюдения по экспериментальным данным

Для MongoDB уже зафиксированы характеристики тестового набора данных: `10000` документов коллекции `objects`, от `12` до `1000` операций в одном документе и среднее значение `507.7832` операций на объект. Это показывает, что документная модель исследуется не только концептуально, но и на заметном объёме данных.

ГЛАВА 4. ИЗМЕРЕНИЯ И РЕЗУЛЬТАТЫ

4.1 Набор команд для анализа

Для обеих СУБД в репозитории сохранены отдельные файлы с фактически использованными командами анализа:

- `postgresql/resources/explain_analyze_commands.md`;
- `mongodb/resources/explain_execution_stats_commands.md`.

Эти файлы фиксируют не только формулировки запросов, но и конкретные параметры, использованные при локальных замерах.

4.2 Результаты PostgreSQL

Для PostgreSQL использовался датасет следующего объёма:

- `operations`: `1,000,000`;
- `objects`: `8`;
- `replicas`: `3`.

Для измерений были выбраны следующие репрезентативные значения:

- `object_id = 'event-7'`: `126,445` операций;
- `replica_id = 'R2'`: `334,323` операций;
- `tx_id = 'R3:tx:238257'`;
- `op_id = 'R3:238257'`.

Сводные результаты измерений для PostgreSQL вынесены в **Приложение А**.

Показательный пример команды для чтения полной истории объекта:

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT
    op_id,
    tx_id,
    object_id,
    replica_id,
    clock,
    action
FROM operations
WHERE object_id = 'event-7'
ORDER BY op_id;
```

Листинг 3 – Пример EXPLAIN ANALYZE для истории объекта в PostgreSQL

Результаты измерений показывают, что PostgreSQL особенно эффективен в точечных и умеренно селективных сценариях. При этом широкие выборки и сортировка больших подмножеств журнала остаются основным источником стоимости.

4.3 Результаты MongoDB

Для MongoDB использовался набор данных со следующими характеристиками:

- количество документов в `objects`: `10000`;
- минимальное число операций в документе: `12`;
- максимальное число операций в документе: `1000`;
- среднее число операций в документе: `507.7832`;
- оценка суммарного числа встроенных операций: примерно `5,077,832`.

Для измерений использовались значения:

- `objectId = "event-3299"`;
- `opId = "R1:365809"`;

- `replicaId = "R5"`;
- `action.type = "field.set"`.

Сводные результаты измерений для MongoDB вынесены в **Приложение Б**.

Показательный пример команды для поиска объекта по вложенному `opId`:

```
db.objects.find(
  { "operations.opId": "R1:365809" },
  { _id: 0, objectId: 1 }
).explain("executionStats")
```

Листинг 4 – Пример explain(”executionStats”) для MongoDB

Полученные результаты подтверждают, что MongoDB лучше всего проявляет себя в объектно-центричных сценариях: поиск объекта, чтение полной истории и доступ к последним операциям выполняются естественно и быстро, тогда как глобальные выборки по операциям остаются менее характерными для этой модели.

ГЛАВА 5. ВАРИАНТ НА YTSAURUS

Раздел по `YTsaurus` в настоящий момент оставлен как заготовка и намеренно не раскрывается содержательно. В дальнейшем сюда планируется добавить модель хранения, сценарии запросов, измерения и сопоставление с PostgreSQL и MongoDB.

ГЛАВА 6. СРАВНЕНИЕ ПОДХОДОВ

Финальное сопоставление вариантов хранения должно учитывать не только различие между PostgreSQL и MongoDB, но и будущий вариант на `YTsaurus`. По этой причине сравнительный раздел размещён после заготовки под третий подход.

На текущем этапе можно сделать следующие предварительные выводы:

- PostgreSQL лучше соответствует модели глобального immutable operation log и аналитических сценариев;
- MongoDB лучше подходит для объектно-центричного доступа, при котором важна полная история конкретного объекта;
- `YTsaurus` пока оставлен как отдельное направление дальнейшего исследования.

Сводное сопоставление трёх подходов вынесено в **Приложение В**.

ЗАКЛЮЧЕНИЕ

В работе рассмотрены два реализованных варианта хранения CRDT-операций: реляционный на PostgreSQL и документный на MongoDB. Для обоих вариантов уже подготовлены окружения запуска, скрипты инициализации, схемы хранения и материалы по анализу типовых запросов.

Сравнение показало, что PostgreSQL лучше соответствует модели глобального журнала операций, тогда как MongoDB удобнее для хранения объектных агрегатов со встроенной историей. Раздел по `YTsaurus` оставлен как направление дальнейшей работы и будет дополнен после подготовки отдельной схемы хранения и экспериментальной части.

ПРИЛОЖЕНИЕ А. СВОДНЫЕ РЕЗУЛЬТАТЫ POSTGRESQL

Таблица 1 – Ключевые результаты EXPLAIN ANALYZE для PostgreSQL

Сценарий	**План**	**Время**	**Наблюдение**
История объекта по `object_id`	`Bitmap Index Scan` + `Parallel Bitmap Heap Scan` + `Gather Merge`	573.7 ms	Индекс по `object_id` используется, но значимая часть стоимости уходит в чтение heap pages и сортировку
Операции транзакции по `tx_id`	`Index Scan` по `idx_operations_tx_id`	0.245 ms	Быстрый точечный сценарий с эффективным использованием B-tree индекса
Операции реплики по `replica_id`	`Parallel Seq Scan` + сортировка	210.0 ms	Индекс не выбран из-за низкой селективности условия
Поиск по `op_id`	`Index Scan` по `operations_pkey`	0.213 ms	Первичный ключ полностью закрывает данный сценарий
Фильтрация по частому `action.type = field.set`	`Parallel Seq Scan`	1004.4 ms	Индекс невыгоден для очень частого значения
Фильтрация по селективному `action.type = set.remove`	`Bitmap Index Scan` + `Parallel Bitmap Heap Scan`	275.4 ms	Индекс по типу действия работает при селективном значении
Поиск по фрагменту `action`	`Bitmap Index Scan` по GIN	485.2 ms	GIN-индекс полезен для containment-запросов по `JSONB`
Поиск по фрагменту `clock`	`Bitmap Index Scan` по GIN	0.534 ms	Индекс по `clock` реально используется и эффективен

ПРИЛОЖЕНИЕ Б. СВОДНЫЕ РЕЗУЛЬТАТЫ MONGODB

Таблица 2 – Ключевые результаты executionStats для MongoDB

Сценарий	**План**	**Показатели**	**Наблюдение**
Поиск объекта по `objectId`	`IXSCAN -> FETCH`	`1 key / 1 doc`	Базовый сценарий полностью поддержан уникальным индексом
Чтение полной истории объекта	`IXSCAN -> FETCH`	`1 key / 1 doc, 2 ms`	История объекта читается как один документ
Поиск по `operations.opId`	`IXSCAN -> FETCH -> PROJECTION`	`1 key / 1 doc`	Multikey индекс работает эффективно
Поиск по `operations.replicaId`	`IXSCAN -> FETCH -> LIMIT`	`100 keys / 100 docs`	Индекс используется корректно, но сценарий вторичен для embedded-модели
Поиск по `operations.action.type`	`IXSCAN -> FETCH -> LIMIT`	`100 keys / 100 docs`	Индекс по типу действия работает корректно
Чтение последних операций через `$slice`	`IXSCAN -> FETCH -> PROJECTION`	`1 key / 1 doc, 4 ms`	Сценарий хорошо соответствует объектной модели

ПРИЛОЖЕНИЕ В. СРАВНЕНИЕ ПОДХОДОВ ХРАНЕНИЯ

Таблица 3 – Сравнение вариантов хранения

Критерий	**PostgreSQL**	**MongoDB**	**YTsaurus**
Единица хранения	Отдельная операция	Документ объекта со встроенной историей	TODO
Физическая структура	Таблицы `replicas`, `objects`, `operations`	Коллекция `objects` и массив `operations`	TODO
Сильная сторона	Глобальный журнал операций, аналитика, ссылочная целостность	Быстрое чтение полной истории объекта	TODO
Слабая сторона	Дорогая сортировка больших выборок, часть структуры в `JSONB`	Ограничение размера документа и менее естественные глобальные выборки	TODO
Инструмент анализа	`EXPLAIN ANALYZE`	`explain("executionStats")`	TODO